Reykjavík University

T-742-CSDA – COMPUTER SECURITY: DEFENCE AGAINST THE DARK ARTS

A Methodology for Low-Level 2.4 GHz Protocol Analysis using the nRF52840

Hafþór Árni Hermannsson
hafthorh20@ru.is

supervised by
Jacky Mallett & TA: Milan Petrusic

November 13, 2025

Contents

1 Introduction **2**

2 Sniffer Design and Implementation **3**
2.1 Initial Sniffer Hardware . . . 3
2.2 Development Environment . . . 3

3 Initial Design Methodology **3**
3.1 Phase 1: The "Preamble Trick" for Promiscuous Capture . . . 3
3.2 Phase 2: Channel Identification via RSSI Sweeping . . . 4
3.3 Phase 3: The Challenge of Parameter Ambiguity . . . 4
3.4 Phase 4: Implementation of an Interactive Tuning Interface . . . 5

4 Analysis and Validation **6**
4.1 Phase 5: Validation with a Known Protocol (Loopback Test) . . . 6
4.1.1 Testbed Setup . . . 6
4.1.2 Diagnostic Journey . . . 7
4.1.3 Solutions and Key Learnings . . . 7
4.1.4 Implications for the M185 Analysis . . . 8
4.2 Phase 6: Revisiting the M185 Target . . . 8
4.2.1 Address Discovery Process . . . 8
4.2.2 Action-Correlated Addresses . . . 8
4.2.3 Persistent Failures: CRC and Replay . . . 8
4.3 Phase 7: Validation of TX/Replay Logic . . . 9
4.3.1 Independent Target Setup . . . 9
4.3.2 The Replay Test . . . 9
4.3.3 Results and Implications . . . 9
4.4 Phase 8: Identifying Bitstream Desynchronization . . . 10
4.4.1 Test Configuration . . . 10
4.4.2 Evidence of Bit-Level Misalignment . . . 10
4.4.3 Cause and Implications . . . 11

5 Project Status and Future Work **11**
5.1 Current Status . . . 11
5.2 Future Work . . . 11

6 Conclusion **12**

Abstract

This report outlines a methodology for reverse-engineering the 2.4 GHz protocol of the undocumented NRF A0 31502F chip found in a Logitech M185 mouse. A custom sniffer tool was developed using the nRF52840's low-level RADIO peripheral. The methodology is presented as a chronological process of discovery. Initial attempts at promiscuous capture using a "Preamble Trick" were hindered by environmental noise, which was solved by implementing an RSSI-based channel sweep. This revealed a deeper problem of protocol ambiguity, leading to the development of an interactive serial-based UI for on-the-fly parameter tuning. When this tool failed to decode the target, a series of validation tests against a known nRF24L01+ protocol were performed. These tests identified that a valid CRC lock was dependent on a combination of non-obvious, low-level registers. Finally, while validating the sniffer's TX replay capabilities, the root cause of the M185's "random" data was identified: **bitstream desynchronization**. This report demonstrates that the Preamble Trick inherently causes a loss of clock synchronization, resulting in corrupted, bit-shifted data. This conclusion reframes the M185 problem as one of bit-level realignment, not (necessarily) encryption, and defines the "autotune" function as the clear next step.

1 Introduction

The primary objective of this project is to extend prior research on the **MouseJack vulnerability** by developing a standalone device capable of sniffing, analyzing, and spoofing the proprietary 2.4 GHz protocols used by modern wireless mice. Many such peripherals remain insecure, often transmitting mouse movements unencrypted, which allows an attacker to inject malicious packets through a seemingly harmless mouse dongle.

As an initial case study, this project targets a **Logitech M185** wireless mouse. This specific mouse is not listed in existing MouseJack vulnerability reports and does not appear to use the well-documented Logitech Unifying protocol. A teardown of the mouse revealed its core component is an undocumented SoC, the **NRF A0 31502F**, likely a custom variant of Nordic Semiconductor's nRF series chips.

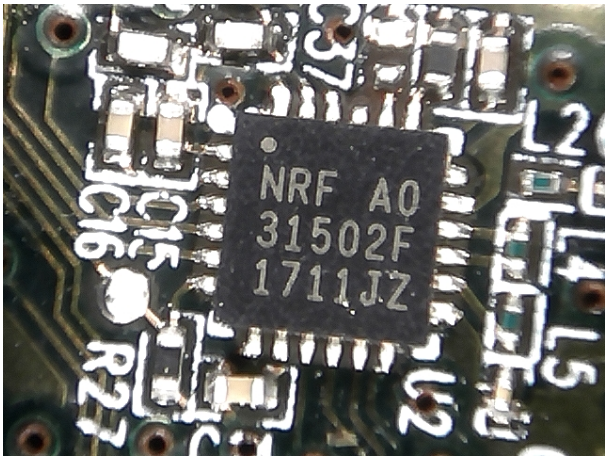

Figure 1: The target NRF 31502F chip on the Logitech M185 PCB

The existing body of public research and open-source tools for this attack almost exclusively relies on the older nRF24L01+ chip coupled with an external microcontroller. These setups typically use hardware like the CrazyRadio PA USB dongle or SparkFun nRF24LU1+ breakout boards for packet sniffing and injection.

Rather than reusing the legacy platforms common in earlier MouseJack work, this project uses the **nRF52840** SoC. Its 2.4 GHz radio is on-air compatible with Nordic's older nRF24L family and exposes a flexible, memory-mapped RADIO peripheral. This enables direct low-level control over modulation, bit rates, packet formats, and timing, as opposed to the nRF24's more rigid SPI-based interface. Important advantages compared to the nRF24 for sniffing and spoofing include:

- True RSSI sampling (1 dB resolution), enabling signal strength-based filtering to isolate target devices (vs. the nRF24's basic binary power detector).
- Larger payloads (up to 255 bytes) with configurable CRC, data whitening, and addressing (up to 8 devices).

- Higher transmit power (+8 dBm max) and receiver sensitivity (-93 dBm at 1 Mbps) for extended range.
- A bit counter for precise bit-level events.
- EasyDMA for efficient, low-CPU packet handling.
- Programmable shortcuts and state machines for reduced latency in custom protocols.

This project leverages these low-level RADIO features, many of which are not exposed in typical nRF24L01-based MouseJack tooling. Since there is no off-the-shelf nRF52840 framework for analyzing proprietary 2.4 GHz mouse protocols, a custom, bit-level sniffer was developed. This level of control is important for exploring how framing, addressing, and CRC configuration affect the interpretation of unknown packets.

2 Sniffer Design and Implementation

2.1 Initial Sniffer Hardware

The initial hardware setup for capturing data from the target mouse was minimal and consisted of two primary components:

- **Sniffer and Injection Tool:** An Adafruit Feather nRF52840 Express. This nRF52840 development board is the core of the project. Its RADIO peripheral is used to capture raw bitstreams (RX) and transmit test packets (TX).
- **Host Controller:** A PC connected to the development board via USB. This machine runs a serial monitor, providing an interactive command interface for controlling the sniffer and logging the captured data.

This basic Feather–plus–host-PC setup is sufficient to reproduce the bulk of the experiments in this report. Additional hardware (such as the external nRF24L01+ modules and the Metro M4 testbed introduced later) is only used to validate and cross-check the sniffer's behaviour against known protocols, not to make the core methodology dependent on specialised equipment.

2.2 Development Environment

The firmware for the nRF52840 was developed in C++ using PlatformIO Core within the Visual Studio Code editor. This environment was chosen over the standard Arduino IDE as its build system and library management are better suited for the low-level register manipulation and debugging required by this project. The firmware was built on the Adafruit nRF52 fork of the Arduino core. the full code is accessible in this github repository

3 Initial Design Methodology

With the sniffer tool prepared, the first step was to capture raw on-air packets without knowing the target's address. This required a method for bypassing the nRF52840's hardware-based address filtering.

3.1 Phase 1: The "Preamble Trick" for Promiscuous Capture

The `NRF_RADIO` peripheral normally requires both a valid preamble and a matching address before it will deliver a packet. To force it into a pseudo-promiscuous mode, the sniffer is configured so that the address-comparison hardware fires as soon as it sees the preamble.

According to the nRF52840 datasheet (sec 6.20.1), the 1-byte preamble for 1/2Mbps (non-BLE) modes is either `0xAA` or `0x55`. By setting the sniffer's address registers to match this (e.g., setting the 1-byte `PREFIX0` to `0x55`), the hardware's address matching logic accepts the preamble as a valid address prefix. Virtually all commercial Nordic-based (and other) 2.4 GHz devices automatically transmit only a 0xAA or 0x55 preamble (chosen by hardware to guarantee a bit-transition for reliable clock recovery), making this preamble-matching trick effective against the overwhelming majority of vulnerable wireless mice and keyboards.

The result, shown in Figure 2, is that the radio captures any packet with a matching preamble. The packet's true address is then (in theory) delivered as the first few bytes in the RAM buffer (`esb_rx_buf`).

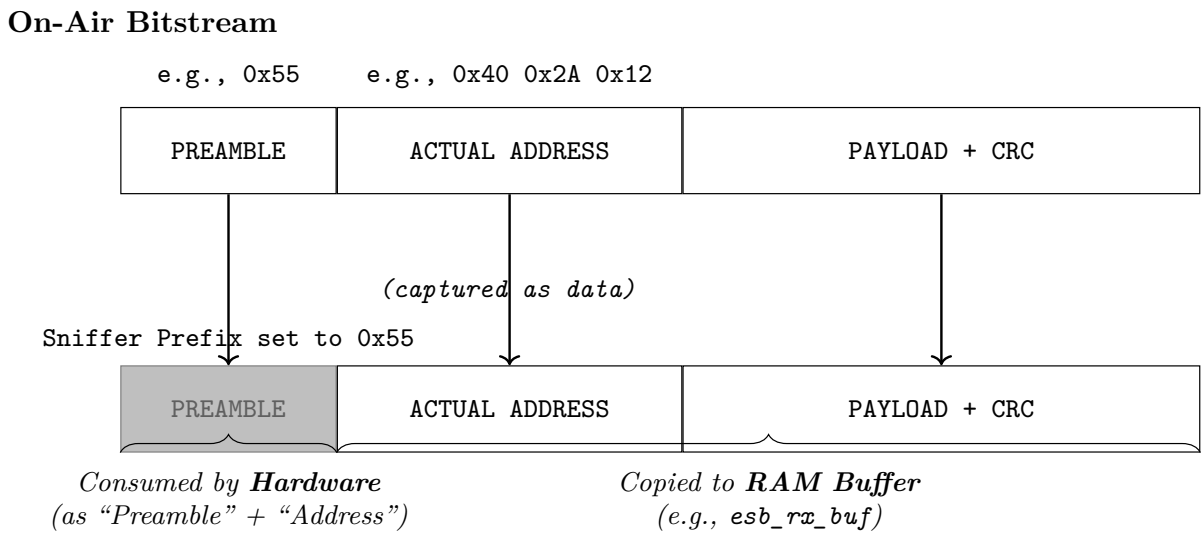

Figure 2: The "Preamble as Address" technique for promiscuous capture. By setting the sniffer's address prefix to match the preamble, the hardware is configured to capture packets, with the true address delivered as the first bytes in the RAM buffer.

3.2 Phase 2: Channel Identification via RSSI Sweeping

The initial implementation of the preamble trick was functionally successful but practically unusable. The 2.4 GHz band is extremely noisy; the sniffer was instantly flooded with a constant stream of random packets from Wi-Fi, Bluetooth, and other devices, making it impossible to isolate the transmissions from the Logitech M185 mouse.

To solve this, a channel sweeping feature with RSSI filtering was implemented. This function, `sweep_once_find_best`, iterates through a specified range of channels (e.g., 2400 MHz to 2500 MHz). On each channel, it dwells for a short period, measuring the Received Signal Strength Indicator (RSSI).

By running this sweep while repeatedly activating the target mouse (e.g., moving it or clicking), it was possible to identify the channel with the strongest and most frequent activity. This allowed the sniffer to be locked to a single channel, filtering out the vast majority of environmental noise and focusing the capture effort on the target device.

3.3 Phase 3: The Challenge of Parameter Ambiguity

Locking onto the target's channel successfully isolated the mouse's packets, but it immediately exposed a more complex problem: parameter ambiguity. The `NRF_RADIO` peripheral is not a simple bit-stream recorder; it actively parses an incoming packet based on its current register configuration (e.g., `PCNF0`, `PCNF1`).

Since the target's packet structure was completely unknown, any parameters set in the sniffer firmware were simply a guess. A single on-air transmission would be captured and presented in memory in vastly different ways depending on these settings. For example, if `PCNF0.LFLEN` was set to 0, the radio would capture a fixed number of bytes defined by `PCNF1.STATLEN`; if set to 8, it would interpret the first byte as a length field and capture a dynamic number of bytes; and if `PCNF1.WHITEEN` was enabled, the hardware would perform a PN9 dewhitening operation on the bitstream before it was even written to RAM.

This ambiguity, illustrated in Figure 3 and Figure 4, made it impossible to determine the correct parameters by simply observing the captured data. The goal was to find a set of parameters that would result in *consistent* and *verifiable* data (e.g., a stable address or a valid CRC).

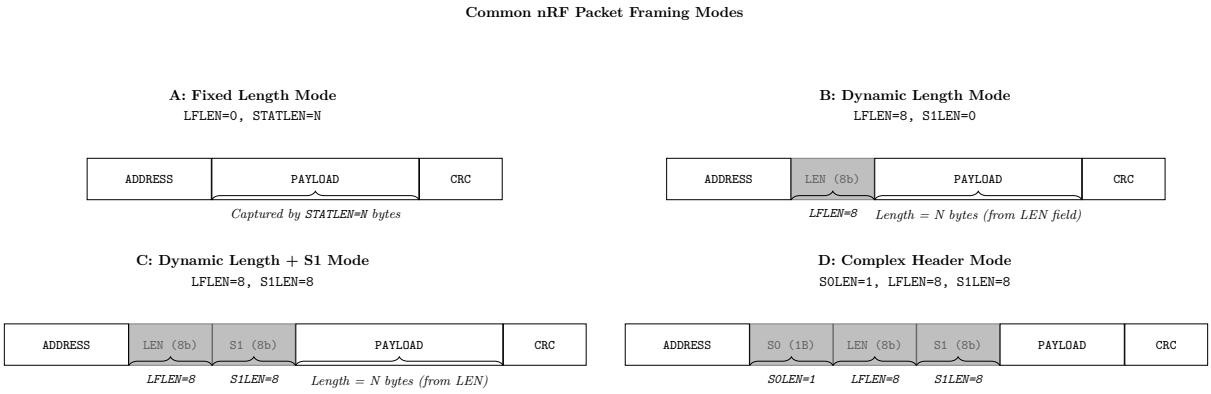

Figure 3: The four most common packet-framing configurations determined by the PCNF0 and PCNF1 registers. The sniffer firmware must guess which, if any, of these structures the target uses.

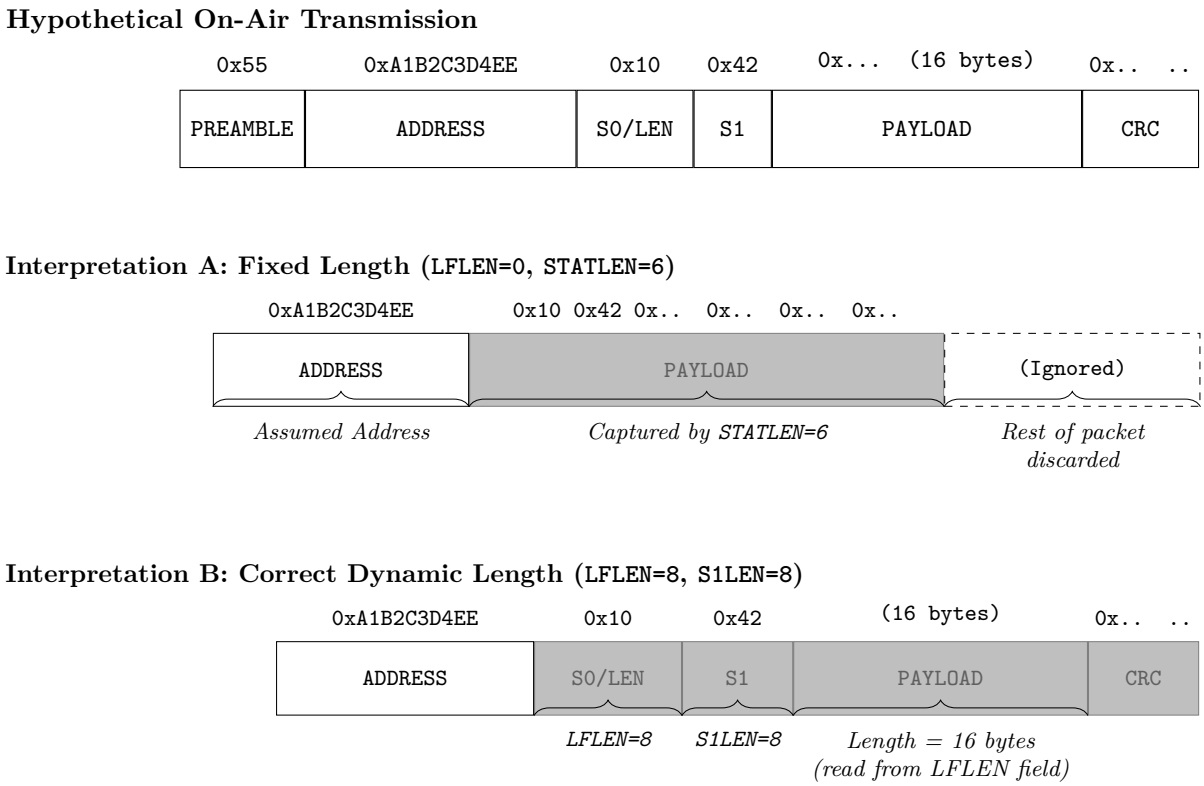

Figure 4: Example of nRF NRF_RADIO parameter ambiguity. The same on-air transmission (Top) is interpreted differently based on the PCNF0 and PCNF1 register settings. Interpretation A (Middle) shows a fixed-length misconfiguration, while Interpretation B (Bottom) shows the correct dynamic-length configuration (Mode C from Figure 3).

3.4 Phase 4: Implementation of an Interactive Tuning Interface

To solve the ambiguity problem, it was necessary to rapidly test hundreds of parameter combinations. Modifying the C++ firmware and re-compiling for every test was unfeasible.

The solution was to expand the firmware to include an interactive user interface controlled via the serial monitor. While the sniffer is in the LISTENING state, this interface accepts single-key commands sent from the host computer. These commands modify the NRF_RADIO peripheral registers in real-time, allowing for immediate observation of how the captured packets change.

This "on-the-fly" tuning made the sniffer much more powerful but is something that later can be automated.

The first interactive commands implemented were:

- **'W'** (Whitening): Toggles `NRF_RADIO->PCNF1.WHITEEN`.
- **'D'** (Data Rate): Toggles `NRF_RADIO->MODE` (1 Mbps / 2 Mbps).
- **'C'** (CRC): Cycles `NRF_RADIO->CRCCNF.LEN` (Disabled, 1-byte, 2-byte).
- **'b, l, s, t, K'**: Adjust packet format registers (e.g., `PCNF1.BALEN`, `PCNF0.LFLEN`, `PCNF0.S1LEN`, `PCNF1.STATLEN`) to test the different packet structures shown in Figure 3.

4 Analysis and Validation

4.1 Phase 5: Validation with a Known Protocol (Loopback Test)

The interactive tuning (Phase 4) failed to produce a stable, repeating packet from the M185 mouse. The captured data remained seemingly random. This raised a question: was the mouse protocol encrypted, or was the sniffer tool itself failing to parse the data correctly?

To solve this, a loopback test was devised. The sniffer would be tasked with capturing packets from a known transmitter that it was also controlling.

4.1.1 Testbed Setup

The hardware for this test, shown in Figure 5, required wiring a second transmitter module to the Adafruit Feather nRF52840 board.

- **Sniffer (Receiver):** The nRF52840's native `NRF_RADIO` peripheral, running the interactive sniffer firmware.
- **Known Transmitter:** An nRF24L01+ module connected via SPI to the nRF52840 board.

The firmware was updated to perform two tasks simultaneously:

1. **Transmit:** Command the nRF24L01+ module (via SPI) to send a repeating payload ("Hello Sniffer!") on Channel 50, at 1 Mbps, with a 5-byte address (`0xE7E7E7E7E7`), 2-byte CRC, and with auto-acknowledgements disabled (`radio.setAutoAck(false)`).
2. **Receive:** Use the nRF52840's native `RADIO` peripheral to try and sniff this known packet, using the interactive tuning interface.

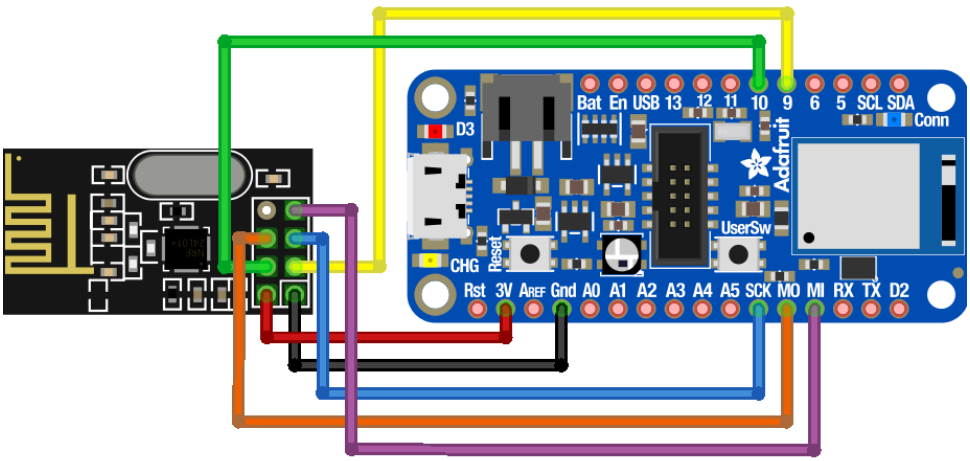

Figure 5: Schematic of the nRF24L01 transmitter connected to the Adafruit Feather nRF52840 sniffer for the loopback validation test.

4.1.2 Diagnostic Journey

Even with all high-level parameters known, the sniffer failed. This began a step-by-step diagnostic process.

Problem 1: The 1-Byte Shift With `LFLEN=6` and `S1LEN=2` (common nRF24 header settings) and CRC disabled, the sniffer captured `65 6C 6C 6F ...` ("ello Sniffer!"). Analysis showed the nRF24's `setAutoAck(false)` setting also disables the 2-bit S1 (PID) field. This was caused by the sniffer, expecting an S1 field (`S1LEN=2`), misinterpreting the first payload byte ('H') as the S1 field. The DMA for the payload then started on the *next* byte ('e'), resulting in the 1-byte shift. The fix was to set `S1LEN=0` in the sniffer UI.

Problem 2: Garbage Data Setting `S1LEN=0` (while keeping `LFLEN=6`) did not fix the problem. The log turned to complete garbage (`19 5B 1B...`). This revealed the root cause: the nRF24 transmitter was sending a completely **headerless packet**. The sniffer, still expecting a 6-bit length field (`LFLEN=6`), was trying to parse the first payload byte ('H', `0x48`) as a length, corrupting the radio's entire packet-parsing state, as shown in Figure 6. The fix was to set `LFLEN=0` in the sniffer UI.

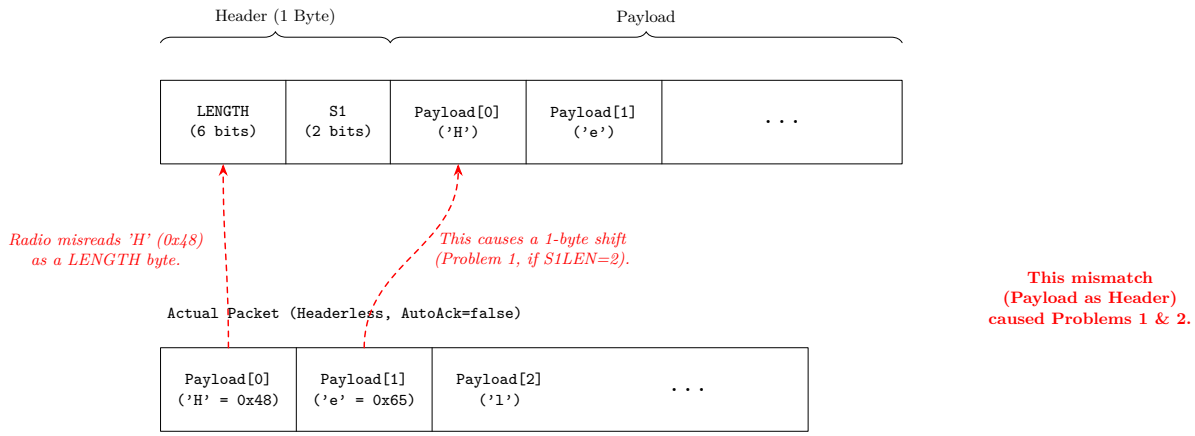

Figure 6: Mismatch between the sniffer's expected ESB packet (with header) and the actual headerless packet transmitted by the nRF24 when `AutoAck=false`.

Problem 3: CRC Validation Dependencies Setting `LFLEN=0` and `S1LEN=0` led to the first successful "Raw" capture. However, getting the hardware to validate the CRC (get a `CRC=OK` message) proved to be more complex. With 2-byte CRC enabled, the sniffer showed a persistent, stream of `CRC=BAD` messages. It was discovered that a successful hardware CRC required several additional low-level parameters to be correct simultaneously. A mismatch in any single one resulted in a failure. The required parameters were:

1. **`STATLEN`:** Had to be set to the payload length (`32`), not the full packet length (`34`).
2. **`ENDIAN`:** The nRF24 is Big Endian, and the nRF52's radio default had to be changed to match.
3. **`CRCINIT`:** The CRC initial value had to be `0xFFFF`.
4. **`CRCPOLY`:** The CRC polynomial had to be `0x1021`.
5. **`SKIPADDR`:** The nRF52 had to be told to include the address in its CRC calculation (set to `0`).

4.1.3 Solutions and Key Learnings

This diagnostic process led to two, successful capture methods and provided some insights for the M185 analysis.

Solution 1: "Raw" Capture (CRC Disabled) This solution treats the packet as a simple, fixed-length stream of bytes. By setting `LFLEN: 0`, `S1LEN: 0`, `CRC: DISABLED`, and `STATLEN: 34` (32-byte payload + 2-byte CRC), a successful capture of the full 34-byte packet was achieved, with the CRC (`C1 85`) visible as the last two bytes of data.

Solution 2: "Protocol-Aware" Capture (CRC Enabled) This solution, where the hardware validates the packet, was only possible after all parameters from "Problem 3" were correctly set. The settings `LFLEN: 0`, `S1LEN: 0`, `CRC: 2 Bytes`, `STATLEN: 32`, `Endian: Big`, `CRCINIT: 0xFFFF`, `CRCPOLY: 0x1021`, and `SKIPADDR: 0` resulted in a clean, 32-byte payload followed by a `CRC=OK` message.

4.1.4 Implications for the M185 Analysis

This validation test proved that the nRF52840 sniffer could capture a known protocol, but only when a complex set of low-level, non-obvious parameters was correct. with even just a single parameter configured wrong, the received packet could look completely different from the true packet.

This discovery provided a new hypothesis for the M185 mouse: The "random" data is not encryption, but a symptom of this same complex parameter dependency. The sniffer is failing because it is missing the correct, non-obvious combination of packet framing, endianness, and CRC polynomial settings.

This meant the interactive UI had to be expanded to control these newly-discovered, critical registers: `ENDIAN`, `CRCPOLY`, `CRCINIT`, and `SKIPADDR`.

4.2 Phase 6: Revisiting the M185 Target

With the new insights from the nRF24L01 loopback test the analysis was refocused on the M185 mouse. The interactive sniffer UI was now a more powerful tool, having been expanded to control the newly-discovered registers: `ENDIAN`, `CRCPOLY`, `CRCINIT`, and `SKIPADDR`.

4.2.1 Address Discovery Process

With the sniffer locked to the mouse's active channel (Phase 2), the foundational address discovery functions were deployed. These are brute-force scanning functions built into the firmware:

1. **‘run_prefix_scan()‘ (‘X‘):** Tests all 256 possible values for the 1-byte address prefix (`NRF_RADIO->PREFIX0`) on a single pipe.
2. **‘run_next_byte_scan()‘ (‘Y‘):** Sequentially brute-forces each byte of the base address (`NRF_RADIO->BASE0`, up to 4 bytes).

These scans led to the hypothesis that the mouse uses a 3-byte address (a 1-byte prefix and a 2-byte base). This was concluded because the `run_next_byte_scan()` function would successfully find the first two bytes of the base address, but a third run of the function (to find a fourth byte) would consistently fail to find any packets.

4.2.2 Action-Correlated Addresses

The scans did not yield a single, stable address. Instead, different 3-byte addresses were consistently observed to correlate with specific mouse actions. For example, some addresses (e.g., `0x80E13E`) would only receive packets during a **left-click**, while others (e.g., `0x806D2B`) would only receive packets when the button was **held down**, and a third set (e.g., `0x40FF17`) correlated to **mouse movement**.

This observation aligns with the Nordic "MultiCeiver" technology, a feature present in documented Nordic chips. The nRF24L01, for instance, can monitor 6 addresses (data pipes) simultaneously, and the nRF52840 (the sniffer chip) can monitor 8.

It is a reasonable hypothesis that the M185's undocumented chip and dongle implement a similar system. This would explain the observed behavior: the dongle could be listening on multiple data pipes at once, and the mouse could be transmitting different report types (e.g., click, move) on specific, pre-defined addresses. This would be a hardware-level method of separating different event types.

4.2.3 Persistent Failures: CRC and Replay

Despite the address discovery, two problems persisted.

1. No CRC Lock At no point did the sniffer report a `CRC=OK` message. Even when locked to one of these "action-specific" addresses and manually cycling through all known, packet framing, CRC, Endian, and Polynomial parameters, the hardware could not validate a single packet.

2. Replay Attack Failure The sniffer's "record and fire" (replay) feature was tested against the real Logitech USB dongle. Packets that were captured from the mouse were re-transmitted, but the USB dongle never responded to or accepted them.

This replay failure presented a new problem: it was unclear if the captured packets were corrupt (due to the `CRC=BAD` issue), or if the sniffer's transmission logic (TX) was flawed. To isolate this variable, a new validation test was required to confirm the sniffer's TX capabilities, which motivated the setup described in the next section.

4.3 Phase 7: Validation of TX/Replay Logic

The replay attack failure on the M185 dongle (Phase 6) made it necessary to validate the sniffer's transmission (TX) logic. A new test was designed to determine if the nRF52840 could successfully transmit and replay a packet to an independent, known receiver.

4.3.1 Independent Target Setup

A separate target was built using an Adafruit Metro M4 and an nRF24L01+ module. The Metro M4 was loaded with a custom firmware (`metro_m4_echo_test.cpp`) that functioned as a simple "echo target."

- **Hardware:** An Adafruit Metro M4, an nRF24L01+ module (with an additional PA/LNA module for optional range testing), and an RGB NeoPixel LED for visual feedback.
- **Firmware Logic:** The Metro M4 firmware was programmed to periodically transmit a 32-byte "magic packet" (`"METRO_ECHO_TEST_01"`) while in a "beacon" state (LED pulsing BLUE). It simultaneously listened for this same packet, and upon a successful reception, it would enter an "echo received" state, turning its LED to solid GREEN.

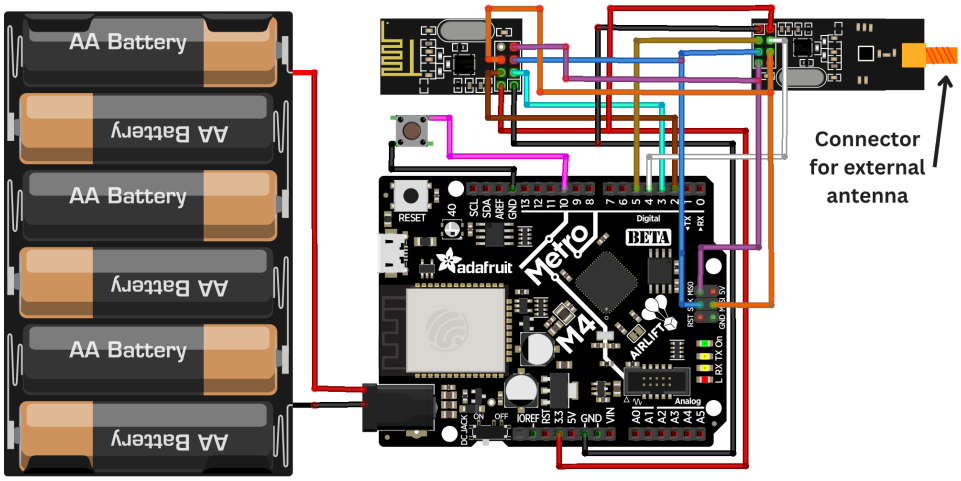

Figure 7: Schematic of the nRF24L01 transmitter/receiver connected to the Metro M4 airlift lite.

4.3.2 The Replay Test

The test was a two-step process:

1. **Capture:** The nRF52840 sniffer was configured with the known-good nRF24L01 parameters (from Phase 5) and used to capture one of the Metro M4's "beacon" packets.
2. **Replay:** The sniffer's "record and fire" (replay) feature was then used to transmit this captured `"METRO_ECHO_TEST_01"` packet back to the Metro M4.

4.3.3 Results and Implications

The test was successful. Upon re-transmitting the packet, the Metro M4's LED turned from BLUE to solid GREEN, indicating it had received and validated the packet from the nRF52840 sniffer.

This test confirmed that the nRF52840 sniffer's transmission (TX) logic was functional. This result suggested the failure to spoof the M185 mouse (Phase 6) was likely not a problem with the sniffer's TX hardware or code, implying that the captured M185 packets were being replayed incorrectly because they were still being captured incorrectly.

4.4 Phase 8: Identifying Bitstream Desynchronization

During the Metro M4 test (Phase 7), a new cause of the M185's "random" data was identified. The observation was made by setting the sniffer to promiscuous mode to capture packets from the known Metro M4 transmitter.

4.4.1 Test Configuration

The sniffer was set to capture packets from the known Metro M4 beacon with the following configuration:

- **Transmitter (Metro M4):** Transmitted a known, fixed packet with these ground-truth settings:
  - Channel: `55`
  - Address: `E7E7E7E7E7`
  - Payload: 32-bytes (`"METRO_ECHO_TEST_01"`)
  - CRC: 2-bytes
- **Sniffer (nRF52840):** Was configured in a "know nothing" promiscuous state:
  - Channel: `55` (found by channel sweep)
  - Address: "Preamble Trick" enabled (`PREFIX0=0xAA`)
  - Packet Config: `LFLEN=0`, `S1LEN=0`, `STATLEN=35`
  - Processing: Whitening disabled, CRC validation disabled.

Under these settings, the sniffer produced a stream of seemingly garbage data:

```
[1964755 ms] Ch 55 pipe=0 RSSI=-28 dBm  ->
  FF FF FB 3E 3F 3F 3F 3F 9F 1F FF 71 92 FF 1F 1F 9E 90
  FF FF 11 7F FE F1 67 60 04 00 00 00 00 00 00 00 00

[1965918 ms] Ch 55 pipe=0 RSSI=-28 dBm  ->
  FE FF FB 3F 3F 3F 3F 3F 9F 1F FF 71 92 FF 1F 1F 9E 90
  FF FF 11 7F FE F1 67 60 04 00 00 00 00 00 00 00 00

[1966416 ms] Ch 55 pipe=0 RSSI=-28 dBm  ->
  FD FF F7 7F 7E 7E 7E 7E 3E 3F FE E3 26 FF 3F 3E 3C 21
  FF FF 23 FE FC E3 CF C1 08 00 00 00 00 00 00 00 00
```

4.4.2 Evidence of Bit-Level Misalignment

This data was not random. The known transmitter address was `0xE7E7E7E7E7`. In binary, `0xE7` is `1110 0111`. If a receiver samples this repeating bitstream at the wrong bit offset, the byte-framing will be incorrect, and it will observe different byte values.

As shown in Table 1, sampling a repeating `0xE7` pattern at different bit offsets produces a predictable set of "corrupted" bytes.

Bit offset	Sampled bits	Observed byte
0	11100111	`E7` (Correct)
1	11001111	`CF`
2	10011111	`9F`
3	00111111	`3F`
4	01111110	`7E`
5	11111100	`FC`
6	11111001	`F9`
7	11110011	`F3`

Table 1: All possible 8-bit windows when sampling a repeated `0xE7` pattern at different bit offsets.

The long runs of bytes observed in the log (e.g., `3F 3F 3F...` and `7F 7E 7E...`) are not random; they are the known `E7` address, viewed through a 3-bit or 4-bit shift.

A second piece of evidence is the long run of zeros at the end of each capture.

```
... F1 67 60 04 00 00 00 00 00 00 00 00
... CF C1 08 00 00 00 00 00 00 00 00 00
```

The known payload was zero-padded. A stream of `0x00` bytes, even when bit-shifted, will still be read as `0x00`. This "stability anchor" confirms the captured data is the known packet, just corrupted by a bit-shift.

4.4.3 Cause and Implications

This desynchronization is a direct consequence of the "Preamble Trick" (Phase 1). The packet preamble is normally used by the radio hardware to synchronize its sampling clock with the incoming bitstream. By configuring the radio to treat the preamble as an "address," this synchronization step is skipped.

This causes the radio's sampling clock to be out-of-sync with the incoming data, resulting in a different, random 0-7 bit offset for each packet capture.

This finding provided a new hypothesis for the M185 data. The "random" packets captured from the mouse (Phase 6) are likely not encrypted. It is far more probable that it is this same bit-shifting phenomenon. This also implies that the "action-correlated addresses" (e.g., `0x80E13E`, `0x40FF17`) found in Phase 6 are themselves bit-shifted artifacts, not the true on-air addresses. This would explain why they failed CRC checks and replay attempts.

This means the captured data must be digitally realigned **before** any further analysis (like CRC brute-forcing) can be successful.

5 Project Status and Future Work

5.1 Current Status

An interactive nRF52840-based sniffer has been developed. Its TX/RX (transmission and reception) logic and the complexity of protocol-aware sniffing have been validated against a known nRF24L01 protocol.

An important finding is that of bitstream desynchronization. This finding suggests that the "random" data captured from the M185 mouse is not necessarily encryption, but is more likely a corrupted, bit-shifted view of the true packets.

The project's current state is that raw data from the mouse can be captured, and the cause of its corruption (bit-shifting) is now understood. The next step is to develop a method to correct this corruption.

5.2 Future Work

The methodology must be extended to correct for this bit-shifting. The planned next steps are to develop an "autotune" function to digitally realign the captured data.

1. **Implement Bitstream Alignment:** Develop a function that operates on the captured, bit-shifted data. This function will, in software, re-slice the raw bitstream at all 8 possible offsets (0-7 bits) to generate 8 "candidate" packets.

2. **Address Reconstruction:** This function will analyze the 8 candidate streams from multiple captured packets. By searching for repeating patterns (like a stable 3-byte address or the "zero-tail" anchor), it will attempt to reconstruct the mouse's true, stable on-air address(es).

3. **Implement On-Chip CRC Brute-Forcing:** Once a valid address is found, the "Preamble Trick" can be disabled. The sniffer will be locked to this valid address, which should re-establish hardware clock synchronization and provide a clean, non-bitshifted stream of packets. At this point, a new sniffer mode will be used to systematically iterate through all hardware configuration parameters until a `CRC=OK` event is triggered.

4. **Differential Analysis:** After a `CRC=OK` lock is achieved, the packet's payload can be analyzed to identify the fields for clicks, movement, and potential rolling codes, which would enable a spoofing attack.

6 Conclusion

The goal of this project was to extend MouseJack-style research beyond legacy nRF24L01-based tooling by building a standalone nRF52840 sniffer capable of analysing undocumented 2.4 GHz protocols at the bit level. The work focused on direct control of its `NRF_RADIO` peripheral and on developing a practical methodology for dealing with unknown packet formats, framing, and CRC parameters.

The resulting toolchain combines three key elements: (1) a promiscuous capture technique based on the "Preamble Trick" and RSSI-driven channel sweeping, (2) an interactive serial UI that allows on-the-fly reconfiguration of critical radio registers, and (3) a validation workflow that uses known nRF24L01+ packets to systematically debug framing, endianness, and CRC assumptions. Through the loopback and Metro M4 echo tests, this methodology demonstrated that the nRF52840 can reliably sniff and replay a known Enhanced ShockBurst protocol, but only when a non-obvious set of low-level parameters is configured correctly.

When applied to the Logitech M185, this methodology did *not* yet yield a fully decoded protocol or a successful spoofing attack. However, the investigation revealed an important insight: the combination of the Preamble Trick and unknown addressing leads to bitstream desynchronization, producing packets that are consistently bit-shifted rather than truly random or demonstrably encrypted. The apparent "garbage" observed from both the Metro M4 beacon and the M185 mouse is best explained as misaligned sampling of otherwise valid packets, and the action-correlated "addresses" are likely artefacts of this misalignment rather than genuine on-air addresses. The clear next step is a software "autotune" stage that realigns captured bitstreams, reconstructs stable addresses, and then uses those addresses to re-enable hardware-based synchronisation and CRC validation. While the M185 protocol itself remains unresolved, the work presented here delivers a reusable methodology for low-level 2.4 GHz protocol analysis with the nRF52840.

References

1 T. Goodspeed, "Promiscuity is the nRF24L01+'s Duty," blog post, Feb. 2011. Available at: http://travisgoodspeed.blogspot.com/2011/02/promiscuity-is-nrf24l01s-duty.html

2 M. Newlin, "MouseJack: Injecting Keystrokes into Wireless Mice," Bastille Threat Research Team Whitepaper, v1.1, Jul. 2016. Available at: https://media.defcon.org/DEF%20CON%2024/DEF%20CON%2024%20presentations/DEF%20CON%2024%20-%20Marc-Newlin-MouseJack-Injecting-Keystrokes-Into-Wireless-Mice-WP-UPDATED.pdf

3 Nordic Semiconductor, "nRF52840 Product Specification," Rev. 1.9 and later, 2017–2024. Available at: https://docs.nordicsemi.com/bundle/nRF52840_PS_v1.9/resource/nRF52840_PS_v1.9.pdf

4 Nordic Semiconductor, "nRF24L01+ Product Specification," Rev. 1.0, 2008 (updated online 2024). Available at: https://docs.nordicsemi.com/bundle/nRF24L01P_PS_v1.0/resource/nRF24L01P_PS_v1.0.pdf